

Programming 1

Orga

- Everyone has a working development environment?
- Everyone has a solid understanding of variables and their types?
- Any questions so far?

Calendar

6./7.10.	Intro, Orga, Hello, World!	8./9.12.	lectures
13./14.10.	Variables, basics	15./16.12.	mid-term
20.21./10.	Collections, Lists, ...	22./23.12.	lectures
27./28.10.	Conditions and Loops	12./13.1.	lectures
3./4.11.	Functions and Modules	18./19.1.	recap
10./11.11.	Classes and Methods	from 26.1.	exams...
17./18.11.	UI + TK basics		
24./25.11.	Spaceship		
01./02.12.	Invaders and Collisions		

Collections, Lists, Dictionaries

Intention

- What if we want to record many items and maybe even don't know the amount before?
 - e.g. the names of the crew on our mission to mars?
- What if names on it's own are insufficient?
 - We also want to record members roles?!
- In today's lecture we will be learning about collections and how to make such tasks possible

Intention

- What if we want to record many items and maybe even don't know the amount before?
 - e.g. the names of the crew on our mission to mars?
- What if names on it's own are insufficient?
 - We also want to record members roles?!
- In today's lecture we will be learning about collections and how to make such tasks possible

```
crew = ["Alice", "Bob", "Charlie"]
```

```
crew_roles = {
    "Alice": "Commander",
    "Bob": "Engineer",
    "Charlie": "Scientist"
}
```

Collections

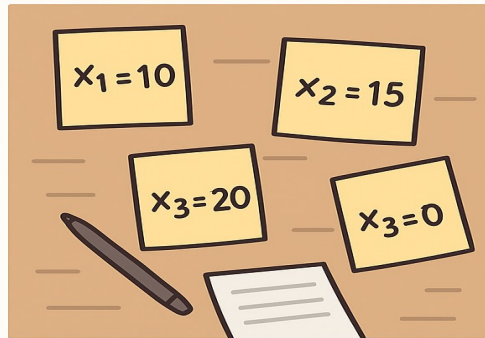
- In python **collections** are **data containers** that are used to store multiple items in a single variable such as:
 - Lists – ordered, mutable collection of items
 - Tuples – ordered, immutable collection
 - Dictionaries – unordered, mutable collection of key – value pairs
 - Sets – Unordered, mutable collection of unique items
- Provided by the collections modules in python (It's part of the standard in built library)

Collections

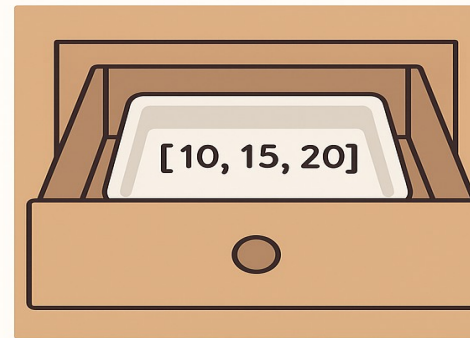
- Why do we use collections?
 - They are used when we want to manage multiple values.
 - When there is a need to iterate, access, sort, group or manipulate these grouped values
 - When there is a need to associate data in a structured way (example: key – value pairs, sequences....)

Illustrative example

- The use of collections can be related to the messy desk vs organized drawer analogy.



Without collections:
track each item manually.



With collections:
manage data in one place.

Collections

- Characteristics of Collections:
 - Size – Can hold any number of elements(including 0)
 - Types – Can hold elements of the same or different types
 - Structure – Can be ordered or unordered
 - Mutability – Some are mutable some are immutable
 - Access Methods – Access by index, key, iteration.

Collections

- Types of Collections:
 - Sequence Collections: Ordered collections that use integer indices like lists, tuples
 - Mapping Collections: Unordered key – value pairs like dict
 - Set Collections: Unordered, unique items only. Sets, Frozen sets
- Choose the right collection based on the goal such as order, if duplicates are allowed or not, if it needs to be changeable.....

Lists

- List : Collection of items in a particular order. In Python, square brackets ([]) indicate a list and individual elements in a list are separated by commas
- Example of a list of fruits is as follows:
 - Fruits = [“apples”, “oranges”, “watermelons”]

Working with Lists

- Accessing elements in a list:
 - Since lists are ordered collections you can access any element in a list by telling Python the position (index) of an element
 - REMEMBER INDEX POSITIONS START AT 0
 - Lists can be indexed forwards:
 - `Lst = [ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 ]` In this list each element is its index value.
 - Lists can also be indexed backwards:
 - `Lst = [-10, -9, -8, -7, -6, -5, -4, -3, -2, -1]`

Indexing in Lists

- Given a list Fruits =
['apple', 'orange', 'watermelon']
 - Fruits[0] would be the value 'apple'
 - Fruits[1] would be the value 'orange'
 - What would Fruits[-1] be?

Indexing in Lists

- Given a list Fruits =
['apple', 'orange', 'watermelon']
 - Fruits[0] would be the value 'apple'
 - Fruits[1] would be the value 'orange'
 - What would Fruits[-1] be?
 - Negative indices count from the end of the list

Modifying Elements in a List

```
1 fruits = ["apple","orange","banana"]
2 print(fruits)
3
4 fruits[0] = "watermelon"
5 print(fruits)
```

- Here we are changing the first element of the list fruits
- We are taking the reference of the first element of the list by using its respective index and assigning it a new value

```
['apple', 'orange', 'banana']
```

```
['watermelon', 'orange', 'banana']
```


Adding Elements to a list

```
1 fruits = ["apple","orange","banana"]  
2 # we want to add watermelon at the start  
3 fruits.insert(0,'watermelon')  
4 print(fruits)
```

```
['watermelon', 'apple', 'orange', 'banana']
```

- We use the `insert()` function to add an element and mention the index and the element to be added

Adding an Element to a list

- Another commonly used function is the `append()` function which allows a user to add an element to the end of a list

```
1 fruits = ["apple", "orange", "banana"]  
2 # we want to add watermelon at the end  
3 fruits.append("watermelon")  
4 print(fruits)
```

```
['apple', 'orange', 'banana', 'watermelon']
```

Removing an element from a List

- The del function is used to delete an item in a list

```
1 fruits = ["apple", "orange", "banana"]
2 del fruits[1]
3 print(fruits)
```

```
['apple', 'banana']
```

Removing an element from a List

- Another function used is called `pop(index value)`. If left empty it would remove and return the last element

```
['apple', 'orange']  
['apple']
```

```
1 fruits = ["apple","orange","banana"]  
2 fruits.pop()  
3 print(fruits)  
4 fruits.pop(1)  
5 print(fruits)
```

List Slicing

- To make a slice the index of the first and last elements that are required are mentioned.
- Python stops one item before the last item mentioned
- Lst[0 : 3] would give an index range of 0,1,2

[1, 2, 3]

```
1 lst = [1,2,3,4,5]
2 print(lst[0:3])
```

Commonly used list functions

- `Len()` - Returns the number of elements in the list
- `Index()` - Returns the index of the object in the list
- `Append()` - Adds an object to the list
- `Extend()` - Add multiple objects of an iterable object to the list
- `Insert()` - Add an object at a given index
- `Count()` - Counts the number of times an object is in a list
- `Reverse()` - returns the same list in a reversed order.

Tuples

- Tuples are similar to lists apart from the fact that they can't be modified. They are immutable
- Example: fruits = ("apple", "oranges", "watermelons")
- Note that once these elements have been assigned they can be accessed by using each items index [EXACTLY LIKE LISTS!!]
- On trying to modify an element we get an error

```
>>> fruits = ("apple", "orange", "watermelon")
>>> fruits[0] = "cherry"
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    fruits[0] = "cherry"
TypeError: 'tuple' object does not support item assignment
```

Writing over a tuple

- How would we solve the above issue?
- We would write over the old tuple

```

fruits = ("apple", "orange", "watermelon")
fruits[0] = "cherry"
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    fruits[0] = "cherry"
TypeError: 'tuple' object does not support item assignment
fruits = ("cherry", "orange", "watermelon")
print(fruits)
('cherry', 'orange', 'watermelon')

```


Writing over a tuple

- Since the name of the tuple is the same we're basically changing where the reference points to.
- The latest defined value of the tuple is used in this case the latest defined value has cherry in the first position.

```

fruits = ("apple", "orange", "watermelon")
fruits[0] = "cherry"
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    fruits[0] = "cherry"
TypeError: 'tuple' object does not support item assignment
fruits = ("cherry", "orange", "watermelon") ←
print(fruits)
('cherry', 'orange', 'watermelon')

```

Modifying an element in a tuple

- Since tuples are IMMUTABLE we can't modify any elements in terms of adding or removing them as seen above
- The only way to change it would require a redefinition of the old tuple. (Creating a new tuple with the same name as the old one)

Useful operations using tuples

```
# packing variables into tuple
tuple1 = 1, 2, "Hello"
# display tuple
print(tuple1)
# Output (1, 2, 'Hello')

print(type(tuple1))
# Output class 'tuple'
```

```
# unpacking tuple into variable
```

```
i, j, k = tuple1
```

```
# printing the variables
```

```
print(i, j, k)
```

```
(i,j,k) = tuple1
```

```
[1,j,k] = tuple1
```

```
# Output 1 2 Hello
```

```
#swapping of variables
```

```
a=1
```

```
b=3
```

Dictionaries

- A Dictionary is a data structure which uses key : value pairs to store items. It is similar to a Dictionary used to look up words
 - {word : meaning} => Dictionary
 - {key : value} => Dictionary in python
- The KEYS must be UNIQUE. No two key : value pairs can have the same key name.
- Example: `eng_deu = {"red" : "rot", "blue" : "blau", "yellow" : "gelb"}`

Ways to create a dictionary

- To create an empty dictionary one can define it so:
 - `E = { }`
 - `E = dict( )`
- To populate the dictionary a person can define key : value pairs during the definition of the dictionary
 - `E = { 1 : "eins", 2 : "zwei", 3: "drei" }`

Modifying values in a dictionary

- To modify a value in a dictionary, the name of the dictionary along with the key in square brackets is assigned a new value

```
E = {1: 'one', 2: 'two', 3: 'three', 4: 'five'}
E[4] = 'four'
print(E)
{1: 'one', 2: 'two', 3: 'three', 4: 'four'}
```

Removing key:value pairs

- When a key : value pair is no longer needed we can use the del statement to completely remove the pair just like how we did it with Modifying

```
E = {1:'one', 2:'two', 3:'three', 4:'five'}
del E[4]
print(E)
{1: 'one', 2: 'two', 3: 'three'}
```


Traversing a dictionary

- Some useful functions to traverse through specific parts of a dictionary are

```
E.keys()
dict_keys([1, 2, 3])
E.values()
dict_values(['one', 'two', 'three'])
E.items()
dict_items([(1, 'one'), (2, 'two'), (3, 'three')])
```

Some important practices

- Using the zip function to create a dictionary
- A zip functions maps one iterable to another.

```
l1 = [1,2,3]
l2 = ["one", "two", "three"]
d1 = zip(l1,l2)
print(d1)
<zip object at 0x000002E68A7C9E80>
print(dict(d1))
{1: 'one', 2: 'two', 3: 'three'}
```

- In the above slide we aren't able to print it directly without type casting it as it belongs to the zip class (We will learn about classes in the upcoming lectures)
- Since we have created an instance with `zip(l1,l2)` we need to type cast it to get the required output

Sets

- A set is an unordered, mutable collection of unique items
- Key points:
 - NO DUPLICATES ALLOWED (important property)
 - Doesn't have index positions since it's unordered
 - Written with { } curly braces
 - Example: numbers = {1, 2, 3}

Creating a set

- `my_set = {1, 2, 3}`
- Using the `set()` constructor:
 - `my_set = set()`
- CAUTION : `emptyt_set = { }` creates an empty dictionary and not a set
- `my_set = set( [ 1, 2, 3, 2, 1] )` #creates a set and removes all the duplicate values.

Creating a set

```
my_set = {2,1,3}
print(my_set)
{1, 2, 3}
set1 = set()
set2 = {}
print(type(set1), type(set2))
<class 'set'> <class 'dict'>
```

Useful functions on sets

- Using the function `add()` - you can add an element to a set
- Using the function `update()` - you can add multiple values
- Using the function `remove()` - you can remove a value, it throws an error if the element you want to remove is missing
- Using the function `discard()` - you can remove a value, it DOESN'T throw an error if the element is missing.

```

fruits = {"apple", "banana"}
fruits.add("cherry")
print(fruits)
{'banana', 'apple', 'cherry'}
fruits.update(["orange", "kiwi"])
print(fruits)
{'cherry', 'banana', 'orange', 'apple', 'kiwi'}
fruits.remove("banana")
print(fruits)
{'cherry', 'orange', 'apple', 'kiwi'}
fruits.discard("pear")
print(fruits)
{'cherry', 'orange', 'apple', 'kiwi'}

```


Set operations

- These operations are very similar to the ones in mathematics
- Union – combine the 2 sets
- Intersection – common items of 2 sets
- Difference – All values belonging to one set and not the other

```
a = {1,2}
b = {2,3}
print(a|b) #union
{1, 2, 3}
print(a&b) #intersection
{2}
print(a-b) #Set difference
{1}
print(b-a) #Set difference
{3}
```

Why use sets?

- Key benefits:
 - Remove duplicates automatically
 - Useful for mathematical set operations
 - Fast membership testing using the keyword “in”

Summary

- You should've gained the following competencies after this :
 - Understand what are collections and why we use them
 - Working with Lists, Tuples, Dictionaries, sets
 - Writing simple programs that enable you to use all the above learned concepts